Find My Next Game

A Steam Game Recommendation App | Project Report

Course: Artificial Intelligence and Data Analytics
Submitted to: Dr. Mindaugas Šarpis
Submitted by: Mantas Bobinas
Date: 26 October 2026

Project overview

For this project, I built an interactive Python application that helps users find Steam games based on their interests and preferences. A user can describe a game they want to play, select a genre and play style, choose a price range in USD, and set minimum review requirements. The app displays up to five recommendations with artwork, descriptions, review information, and links to the Steam store.

I began with no experience using Python or GitHub. The project developed from a simple Python greeting into a working Streamlit app. I used Windows Command Prompt and Notepad, with ChatGPT assistance for explanations, code suggestions, debugging, and interface design. I ran the code locally and checked the results while learning how each part worked.

Main work done

I created a GitHub repository, installed Python and Git, and set up a virtual environment to keep the project's libraries separate. I learned to stage files, create commits, push changes, and integrate remote changes. The project includes setup instructions in a README and dependency versions in requirements.txt.

The first app filtered a 100-record sample by genre and budget, then ranked games by total review count. I added positive-review percentages and minimum review thresholds before expanding to the full dataset. Next, I introduced text matching using game descriptions, genres, and tags. The interface was redesigned with a dark theme, sidebar filters, game cards, and Steam links. Price-range and play-style controls were added afterward.

Dataset and preparation

The source is the FronkonGames Steam Games Dataset on Kaggle [1]. The downloaded CSV contains **125,855 records**. Its header initially had 39 fields because **DiscountDLC count** combined two names. The loader separates it into **Discount** and **DLC count** in memory. A full-file check confirmed that every record contains 40 values; the original CSV is left unchanged.

One record with a missing game name was excluded, leaving **125,854 games**. Checks found no duplicate AppIDs and no missing, non-numeric, or negative prices or positive/negative review counts. There were 8,423 missing genre values and, among the named games, 8,965 missing category values. Such games remain eligible under unrestricted filters but cannot match a specific missing genre or play style. Missing descriptions and tags are treated as empty text.

Recommendation method and tool choices

pandas handles the CSV and filtering, **Streamlit** provides the interface, and **scikit-learn** provides the text-matching tools. These choices allow the project to run locally with an explainable method and no paid AI API. Loaded data and the fitted search index are cached so they can be reused between interactions.

With no written request, eligible games are ranked by total review count. With a request, the app combines each game's genres, tags, and description. **TF-IDF** represents these texts as weighted word features, removes a predefined list of common English words, and limits the vocabulary to 20,000 terms. **Cosine similarity** compares the request with each game. Games with zero similarity are excluded; the rest are ranked by similarity, with review count breaking ties.

All selected filters must be satisfied before the final five games are displayed. Positive-review percentage is calculated as positive reviews divided by total reviews, multiplied by 100, and rounded to one decimal place. Games without reviews have an unavailable percentage. Co-op filtering uses explicit category labels, including online, LAN, and shared/split-screen co-op.

Outcomes and evaluation

The resulting app provides text search, genre and play-style selection, minimum and maximum prices, review thresholds, and clickable store links. Manual checks covered loading the full dataset, applying filters, displaying review percentages, opening a Steam link, and searching for farming and civilization-building games.

One comparison kept the filters fixed at all genres, a $20 maximum, at least 80% positive reviews, and at least 100 reviews. Without text, the results included Counter-Strike 2, Dota 2, Grand Theft Auto V Legacy, Terraria, and Rainbow Six. Searching for **"relaxing farming crafting"** instead returned four Farming Simulator titles and a Doraemon Story of Seasons title. This shows a shift toward farming-related results, but does not establish that every game meets all three requested qualities.

This was an informal comparison, not a systematic accuracy study. No precision score or user-satisfaction rate was measured. Four recommendations from one series also showed limited variety. A stronger evaluation would use several fixed requests, judge relevance against consistent criteria, and compare both ranking methods.

Limitations, reproducibility, and learning

TF-IDF matches vocabulary rather than fully understanding meaning, so synonyms and requests such as "no combat" may be handled poorly. Missing metadata can weaken results. Prices are historical USD values; a recorded zero should not be treated as verified evidence that a game is currently free. Ratings and availability may also change.

To reproduce the app, create and activate a Python virtual environment, install requirements.txt, place the extracted CSV at data/games.csv, and run **python -m streamlit run app.py**. Development used Python 3.15.1. The exact dataset version is not yet recorded, so future downloads may differ. GitHub holds the code and README; the large CSV is maintained separately. Temporary inspection scripts supported development, with their earlier versions retained in Git history.

I learned how variables, functions, indentation, data tables, missing values, filters, and text features work together. The project addresses the course aims through reasoned tool choices, documented setup, automated data preparation and ranking, and organised files with version history.